

Microsoft GraphRAG avec un Knowledge Graph RDF

Partie 3 - Réécriture technique fidèle en français

Sujet de l'article: utiliser SPARQL et le Knowledge Graph pour alimenter le RAG.

Article source: Ian Ormesher, "Microsoft GraphRAG with an RDF Knowledge Graph - Part 3", publié le 24 août 2024 sur Medium.

Document produit le 27 juin 2026. Cette version n'est pas une copie intégrale: elle reformule fidèlement les étapes et les constats techniques vérifiables.

Portée

Cette fiche reprend la troisième partie de la série. La première partie couvrait l'exécution locale de Microsoft GraphRAG avec un LLM et un modèle d'embeddings. La deuxième partie montrait le chargement des sorties GraphRAG dans un magasin RDF, avec GraphDB et un index vectoriel Elasticsearch. La troisième partie assemble ces éléments pour répondre à une question en exploitant à la fois la recherche vectorielle, SPARQL, le graphe RDF et un LLM local.

Principe de prudence: lorsque l'article renvoie à des captures de code ou à des notebooks, ce document décrit l'intention et les objets manipulés sans reconstituer du code exact non lisible dans l'extraction textuelle.

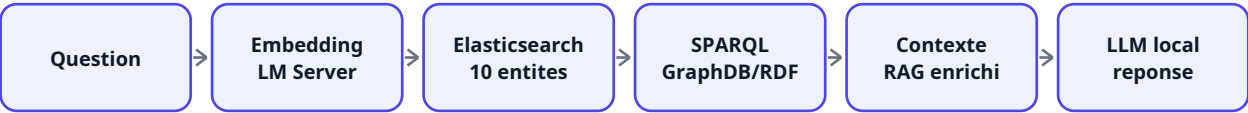
1. Identification de l'article

Élément	Information vérifiée
Titre	Microsoft GraphRAG with an RDF Knowledge Graph - Part 3
Sous-titre	Using SPARQL and the Knowledge Graph for RAG
Auteur	Ian Ormesher
Date	24 août 2024
Durée indiquée	5 minutes de lecture
Objet	Utiliser SPARQL, un magasin RDF, Elasticsearch, LM Studio/LM Server, LangChain et un LLM local pour

L'article s'inscrit dans une série de trois billets. La partie 3 suppose que le Knowledge Graph est déjà peuplé dans un magasin RDF. L'auteur dit utiliser GraphDB comme RDF store. La partie 3 ne refait donc pas l'indexation GraphRAG initiale: elle se concentre sur l'interrogation du résultat.

2. Objectif technique de la partie 3

L'objectif est de répondre à une question utilisateur en construisant un contexte plus riche que celui d'un RAG vectoriel classique. La question est d'abord transformée en vecteur d'embeddings. Ce vecteur permet de trouver les entités les plus proches dans l'index Elasticsearch créé précédemment. Ensuite, les identifiants des entités trouvées servent de points d'entrée vers le Knowledge Graph RDF. SPARQL est utilisé pour extraire les chunks associés, les communautés associées, les relations internes et externes, ainsi que les descriptions des entités. Ces données sont transformées en contexte textuel, puis envoyées à un LLM local via LangChain.



Ce document reformule la logique du billet: la recherche vectorielle ne produit pas directement la réponse finale. Elle sert plutôt à choisir un ensemble initial d'entités, puis le graphe RDF sert à élargir et structurer le contexte.

3. Déroulement du pipeline

3.1 Encodage de la question

La première étape consiste à encoder la question utilisateur avec l'endpoint d'embeddings fourni par LM Server. L'article précise que cet endpoint est compatible avec le style OpenAI. Le résultat est un vecteur de nombres flottants dont la dimension dépend du modèle d'embeddings retenu. Dans l'exemple de l'auteur, le modèle bge-large-en-v1.5-gguf produit des vecteurs de taille 1024.

- Le même modèle d'embeddings doit être utilisé pour la question et pour les données indexées.
- Si le modèle change entre l'indexation et l'interrogation, les distances dans l'espace vectoriel cessent d'avoir le sens attendu.
- L'article ne présente pas cette étape comme une option: elle est nécessaire pour relier la question au même espace vectoriel que les entités.

3.2 Recherche des entités les plus proches

Le vecteur de la question est envoyé à l'index Elasticsearch mis en place dans la partie 2. L'index concerne les entités et leur champ d'embeddings de description. L'objectif est de récupérer les dix entités les plus proches de la question dans l'espace vectoriel.

Sortie de cette étape: une liste d'identifiants d'entités. Cette liste est ensuite utilisée comme point d'ancrage pour les requêtes SPARQL.

3.3 Requêtes SPARQL sur le Knowledge Graph RDF

L'article enchaîne ensuite plusieurs requêtes sur le graphe RDF. La logique n'est pas de se limiter aux textes les plus proches, mais de récupérer les objets liés aux entités trouvées. Les requêtes visent principalement quatre familles d'informations:

- Les chunks associés aux entités, puis classés selon le nombre d'entités retrouvées qui leur sont liées.
- Les communautés associées aux entités, puis triées notamment selon des critères de rang et de poids.
- Les relations entrantes et sortantes autour des entités, via la relation `related_to`.
- Les descriptions des entités, afin de donner au LLM un contexte sémantique lisible.

L'article indique que les trois meilleurs chunks et les trois meilleures communautés sont retenus dans cette logique de récupération.

4. Construction du contexte RAG

Les résultats SPARQL sont manipulés sous forme de DataFrames. Avant d'interroger le LLM, ces DataFrames doivent être convertis dans un format textuel qui peut être inclus dans le prompt. L'article illustre cette conversion avec un DataFrame d'entités, puis applique le même principe aux autres résultats: chunks, communautés, relations et descriptions.

Le point important est que le contexte transmis au LLM ne provient pas d'un seul mécanisme. Il combine:

- une amorce vectorielle, obtenue par similarité sur les descriptions d'entités;
- une expansion graphique, obtenue par SPARQL dans le Knowledge Graph RDF;
- des fragments textuels, via les chunks;
- des synthèses ou regroupements, via les communautés;
- des relations explicites, qui rendent possibles des inférences simples entre entités.

5. Utilisation de LangChain et du LLM local

Une fois le contexte construit, l'auteur configure un chat avec l'instance locale de LM Server. Il mentionne que LM Server fournit du code d'exemple pour se connecter à son serveur, et il utilise ensuite LangChain pour enchaîner le modèle de chat et la génération de réponse.

Le rôle de LangChain, dans cette partie de l'article, est d'organiser l'appel au modèle et la construction de la réponse à partir de la question et du contexte. Le billet ne présente pas LangChain comme un composant RDF ni comme un moteur de graphe: il est utilisé comme couche d'orchestration du prompt et de l'appel au modèle local.

Aucune revendication de performance générale n'est démontrée ici. L'article montre plutôt un prototype fonctionnel qui combine un LLM local, un index vectoriel, un RDF store et des requêtes SPARQL.

6. Exemple de validation: Bob Cratchit et Belinda Cratchit

L'auteur teste le système avec une question portant sur la relation entre Bob Cratchit et Belinda Cratchit dans *A Christmas Carol*. Il explique avoir choisi volontairement une question où le nom complet n'apparaît pas directement dans le texte, mais où la relation peut être déduite en exploitant les liens.

6.1 Sans contexte du Knowledge Graph

Lorsqu'il interroge le modèle sans le contexte extrait du graphe, la réponse est incorrecte. Le modèle semble s'appuyer sur sa connaissance générale de l'œuvre et produit une explication qui ne résout pas correctement la relation attendue. L'auteur utilise cet échec pour montrer que la connaissance interne du modèle, ou un contexte insuffisant, ne suffit pas dans ce cas.

6.2 Avec contexte issu du graphe

Quand la même question est posée avec le contexte récupéré par la combinaison embeddings, Elasticsearch, SPARQL et RDF, la réponse devient correcte dans l'expérience rapportée: Belinda Cratchit est identifiée comme la fille de Bob Cratchit. Le système est aussi capable d'ajouter une relation familiale plus large à partir du contexte fourni.

Ce que l'expérience montre: le graphe fournit au modèle des relations explicites et des descriptions qui lui permettent de répondre à une question nécessitant un raccord entre entités. Ce que l'expérience ne prouve pas: elle ne constitue pas un benchmark statistique, ni une garantie que toutes les questions relationnelles seraient bien résolues.

7. Lecture technique de l'apport RDF/SPARQL

Dans cette architecture, RDF et SPARQL ne remplacent pas le LLM ni la recherche vectorielle. Ils ajoutent une couche déclarative et structurée permettant de naviguer dans les entités, les chunks, les communautés et les relations. Le moteur vectoriel sert à trouver de bons points de départ; le magasin RDF sert ensuite à faire l'expansion relationnelle et à donner une forme contrôlable au contexte.

8. Architecture reconstruite fidèlement

Composant	Rôle dans la partie 3
Question utilisateur	Point de départ de la recherche. Elle est transformée en vecteur.
LM Server / LM Studio	Fournit l'endpoint d'embeddings et l'endpoint de chat compatible avec l'approche OpenAI.
Modèle d'embeddings	Transforme la question en vecteur dans le même espace que les entités indexées.
Elasticsearch	Recherche les dix entités les plus proches de la question.
GraphDB / RDF store	Contient le Knowledge Graph RDF alimenté dans la partie 2.
SPARQL	Récupère chunks, communautés, relations et descriptions liées aux entités trouvées.
DataFrames	Servent de structure intermédiaire avant transformation en contexte textuel.
LangChain	Orchestre l'appel au modèle de chat avec la question et le contexte.
LLM local	Produit la réponse finale enrichie par le contexte du Knowledge Graph.

9. Limites explicitement importantes

- Le billet montre une expérience locale et un exemple de question; il ne présente pas une évaluation systématique.
- La réponse finale dépend du modèle local utilisé, de ses paramètres, du prompt, du contenu RDF et de la qualité des embeddings.
- Le code détaillé est référencé dans le dépôt GitHub; certaines portions de l'article apparaissent sous forme de captures ou d'exemples non entièrement lisibles dans l'extraction textuelle.
- L'index vectoriel est extérieur au RDF store dans cette démonstration: Elasticsearch est utilisé pour la similarité vectorielle, puis GraphDB/SPARQL pour l'expansion dans le graphe.
- Le choix du même modèle d'embeddings entre indexation et interrogation est une contrainte forte.

10. Ce que cette partie apporte à la série

Les parties 1 et 2 préparent les artefacts: extraction GraphRAG, fichiers de sortie, ontologie, RDF store et index vectoriel. La partie 3 montre comment utiliser ces artefacts au moment de la question. C est donc la partie qui transforme le Knowledge Graph RDF en composant actif du RAG.

L apport principal est la combinaison de deux formes de recherche: une recherche vectorielle pour amorcer la sélection des entités, puis une recherche structurée SPARQL pour récupérer les relations et le contexte du graphe. Cette combinaison permet à l auteur d obtenir une réponse correcte sur une relation qui n était pas correctement traitée sans le contexte du Knowledge Graph.

11. Mise en perspective pour un GraphRAG sémantique

La série est intéressante pour une approche GraphRAG fondée sur les technologies du Web sémantique, car elle montre une passerelle pratique entre Microsoft GraphRAG et RDF. Cependant, la solution décrite n est pas encore un moteur GraphRAG RDF complet et autonome. Elle reste une intégration composée de plusieurs briques: Microsoft GraphRAG pour créer les sorties, un notebook pour transformer et charger, GraphDB pour le RDF, Elasticsearch pour les vecteurs, LM Server pour embeddings et chat, et LangChain pour l orchestration.

Dans une architecture industrielle, le point à étudier serait la réduction du nombre de briques, la gouvernance des ontologies, la gestion de provenance, la reproductibilité des requêtes SPARQL, la sécurité des endpoints et l intégration éventuelle d embeddings directement au niveau du serveur de graphe ou d une couche de service spécialisée.

12. Sources utilisées

Article Medium - Partie 3

<https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-3-328f85d7dab2>

Dépôt GitHub associé

https://github.com/ianormy/msft_graphrag_blog

Documentation Microsoft GraphRAG

<https://microsoft.github.io/graphrag/>

Microsoft Research - GraphRAG sur GitHub

<https://www.microsoft.com/en-us/research/blog/graphrag-new-tool-for-complex-data-discovery-now-on-github/>

Projet Microsoft Research GraphRAG

<https://www.microsoft.com/en-us/research/project/graphrag/>

Note de conformité: ce document ne reproduit pas l article complet. Il en fournit une réécriture technique fidèle, fondée sur les éléments accessibles et vérifiables, en évitant d inventer du code ou des résultats non présents dans les sources.